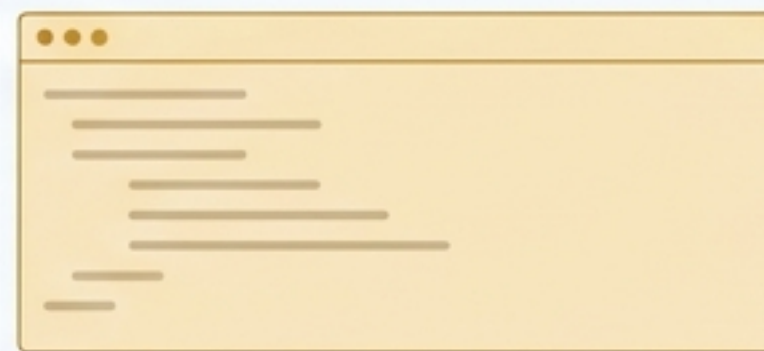


Apache Hive Masterclass

Advanced Data Structures, Query Optimization,
and Distributed Joins

```
select Hive {  
  contest,  
  query: vxst (  
    name: cacotory join(lisxst,n),  
    length:n  
  )  
  return hive;  
}
```



```
select Nive {  
  sontilist,  
  compax-ovs=select (  
    id> {0e0},  
    separator: {W80S,I...92V0},  
    end.acm-n=as STRET,  
  )  
}
```

```
> execute hive_masterclass.sh
```

Prerequisite Recap: The 7 Optimizations

- ✓ Partitioning
- ✓ Bucketing
- ✓ Vectorization
- ✓ Cost-Based Optimization (CBO)
- ✓ Indexing
- ✓ File Formats
- ✓ Apache Tez Engine

Primitive Types

TINYINT,
SMALLINT, INT,
BIGINT, FLOAT,
DOUBLE, DECIMAL,
STRING,
BOOLEAN,
TIMESTAMP

Advanced Data Types

ARRAY,
MAP,
STRUCT,
UNION

While INT and STRING power basic tables, handling nested, complex datasets requires Hive's structural engineering types.

Advanced Data Type: ARRAY



Collection of elements with the SAME data type.

Raw Data

1, Math, Science, Physics

1, Math, Science, Physics

Hive Table DDL

```
CREATE TABLE student (  
    ...  
    subject ARRAY<STRING>  
)  
ROW FORMAT DELIMITED  
FIELDS TERMINATED BY ','  
COLLECTION ITEMS  
TERMINATED BY ',' ;
```

Query Execution

```
SELECT student_id,  
       subject[0],  
       subject[1]  
FROM student;
```

Math, Science

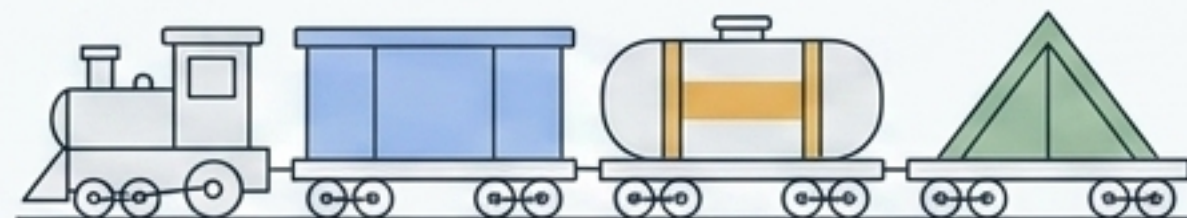
Advanced Data Type: MAP



Collection of
Key-Value pairs.



Advanced Data Type: STRUCT



Combination of DIFFERENT data types within a single record.

Stage 1 Raw Data

2,Raj|25|Pune

Stage 2 Hive Table DDL

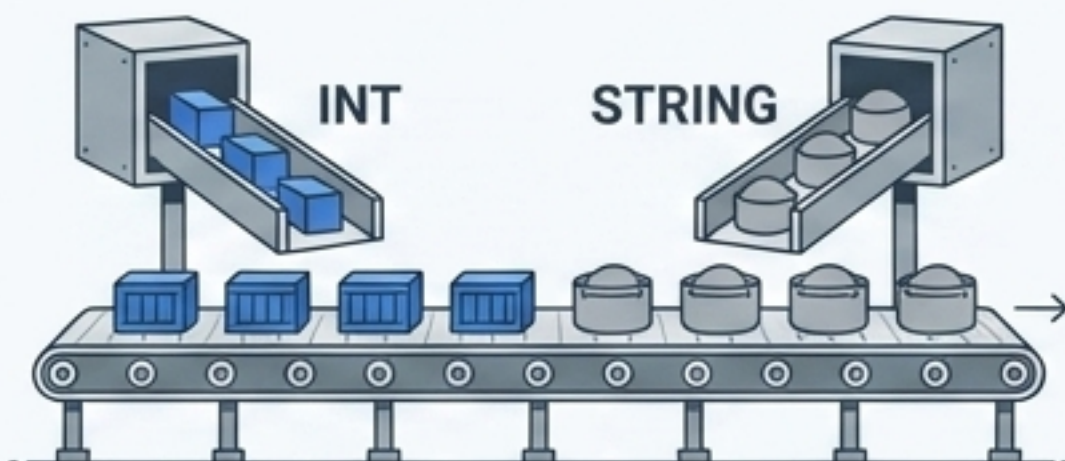
```
employee_info
STRUCT<name:STRING,
age:INT, city:STRING>
...
COLLECTION ITEMS
TERMINATED BY '|';
```

Stage 3 Query Execution

```
SELECT emp_id,
employee_info.name,
employee_info.city
FROM employee;
```

2, Raj, Pune

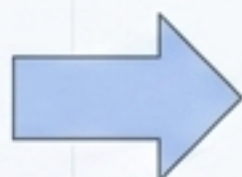
Advanced Data Type: UNION



Stores exactly one data type from a specified list of types per row.

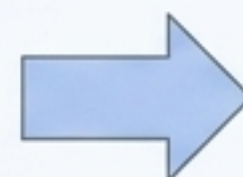
Raw Data

```
80|Pune  
50|Mumbai
```



Hive Table DDL

```
student_details  
UNIONTYPE<INT, STRING>  
... COLLECTION ITEMS  
TERMINATED BY '|';
```



Query Execution

```
SELECT  
  student_details.tag_0,  
  student_details.tag_1  
FROM sample_union;
```

Tag Routing: Hive uses zero-indexed tags (**tag_0** for the integer score, **tag_1** for the string **city**) to automatically route elements of a union type when queried.

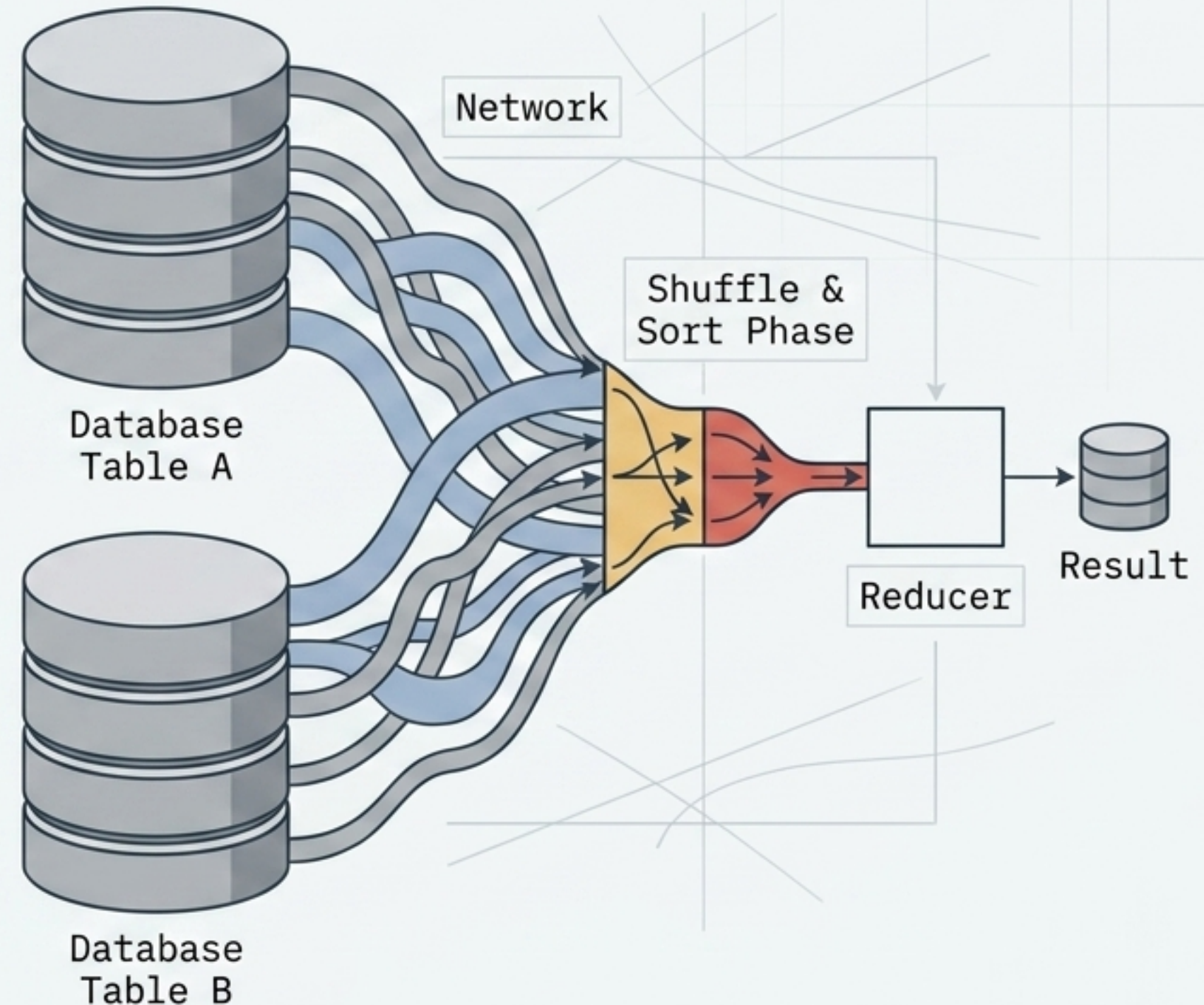
Synthesis: Advanced Data Types Diagnostic

Interview Focus: Expect questions asking you to choose between Struct (known schema/types) vs Map (dynamic keys/uniform values).

Data Type	Definition	Access Syntax	Key Differentiator
ARRAY	Collection of SAME data types	<code>column_name[index]</code>	Homogeneous lists.
MAP	Key-Value pairs	<code>column_name['key']</code>	Dictionary-style lookups; requires MAP KEYS TERMINATED BY.
STRUCT	Collection of DIFFERENT data types	<code>column_name.property</code>	Groups related heterogeneous attributes into a single record.
UNION	Stores exactly one type from a specified list	<code>column_name.tag_x</code>	Row-level variance handling.

Advanced Joins: Reduce-Side Join

- **The Default State:** This is the default join in Hive. No optimizations applied.
- **The Bottleneck:** Massive data from both tables is shuffled across the network to the Reducer.
- **The Cost:** Highly expensive due to excessive Network I/O and Disk I/O.
- **Verdict:** Exceptionally slow for large datasets; this is the baseline bottleneck we aim to optimize.



Advanced Joins: Map-Side Join

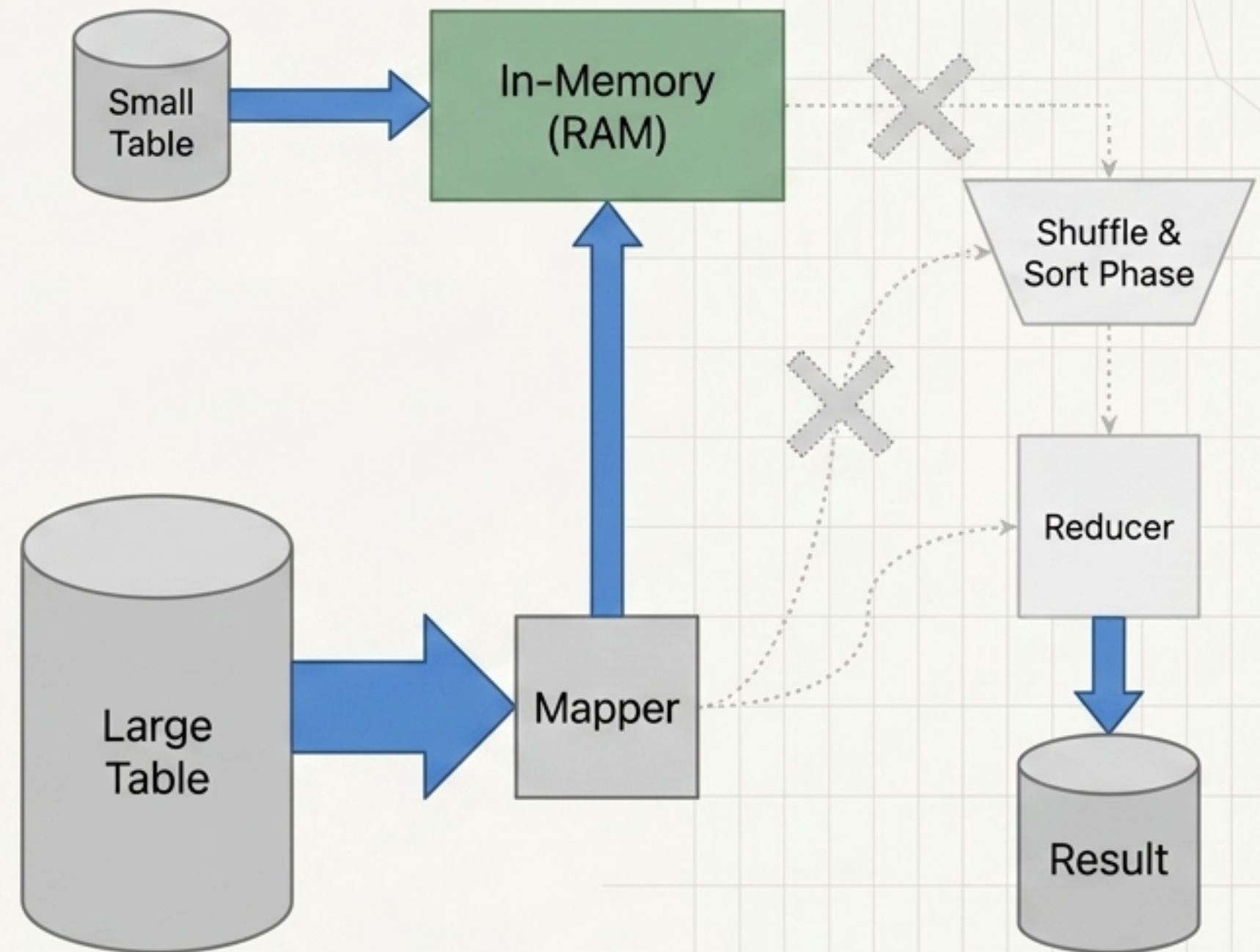
Requirement: One table MUST be small enough to fit entirely in memory.

Execution: The join happens purely in the Mapping phase. No reducer is required, completely bypassing shuffle overhead.

Configuration Snippet:

```
SET hive.auto.convert.join=true;
```

Warning: If both tables are large, Map-Side join will fail entirely due to OutOfMemory (OOM) errors.



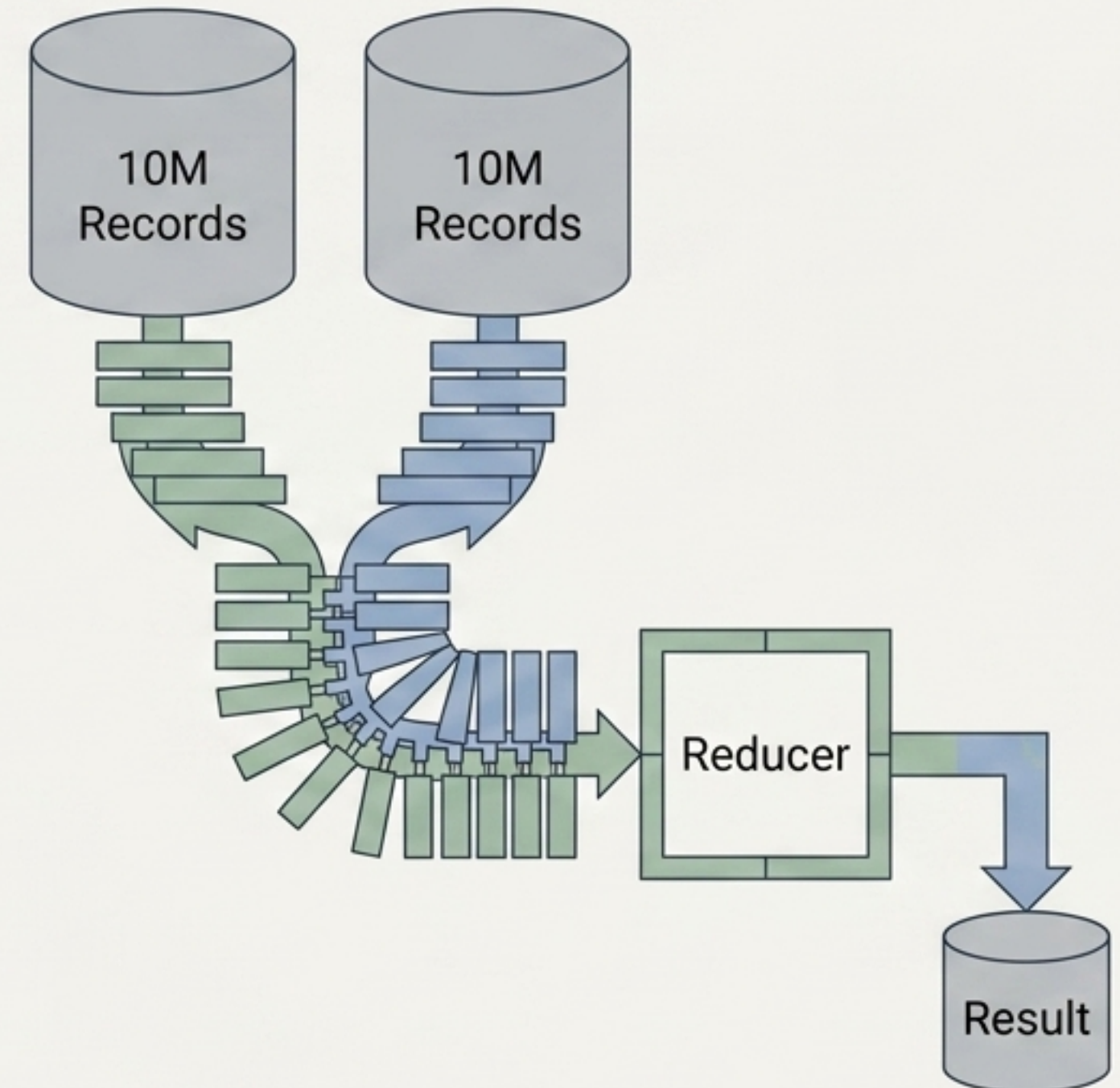
Advanced Joins: Sort-Merge Join

Use Case: Both tables are heavily populated (e.g., millions of records) making Map-Side join impossible.

Execution: Both tables must be pre-sorted (ascending/descending) on the join key. The join occurs in the Reducer, but with drastically reduced shuffle friction because the sorting is already handled.

Properties to Enable:

```
SET hive.enforce.sortmergebucketmapjoin=false;  
SET hive.optimize.bucketmapjoin=true;
```



Advanced Joins: SMB & Skew Join Optimizations

Before

Unoptimized Join on Skewed Data



3 Hours

After

Sort-Merge Bucket (SMB) Join



1 Hour

The Concept: SMB Join applies the Sort-Merge logic specifically to Bucketed tables. Data is divided into corresponding buckets, allowing mappers to join matching buckets directly.

Requirement: Both tables must be clustered and sorted into the exact same number of buckets.

Skew Handling: Skew joins solve for the specific problem where certain keys contain disproportionately huge amounts of data, causing severe reducer bottlenecks.

Synthesis: The Distributed Joins Diagnostic

Top Interview Question: Difference between Map-Side and Sort-Merge Joins.

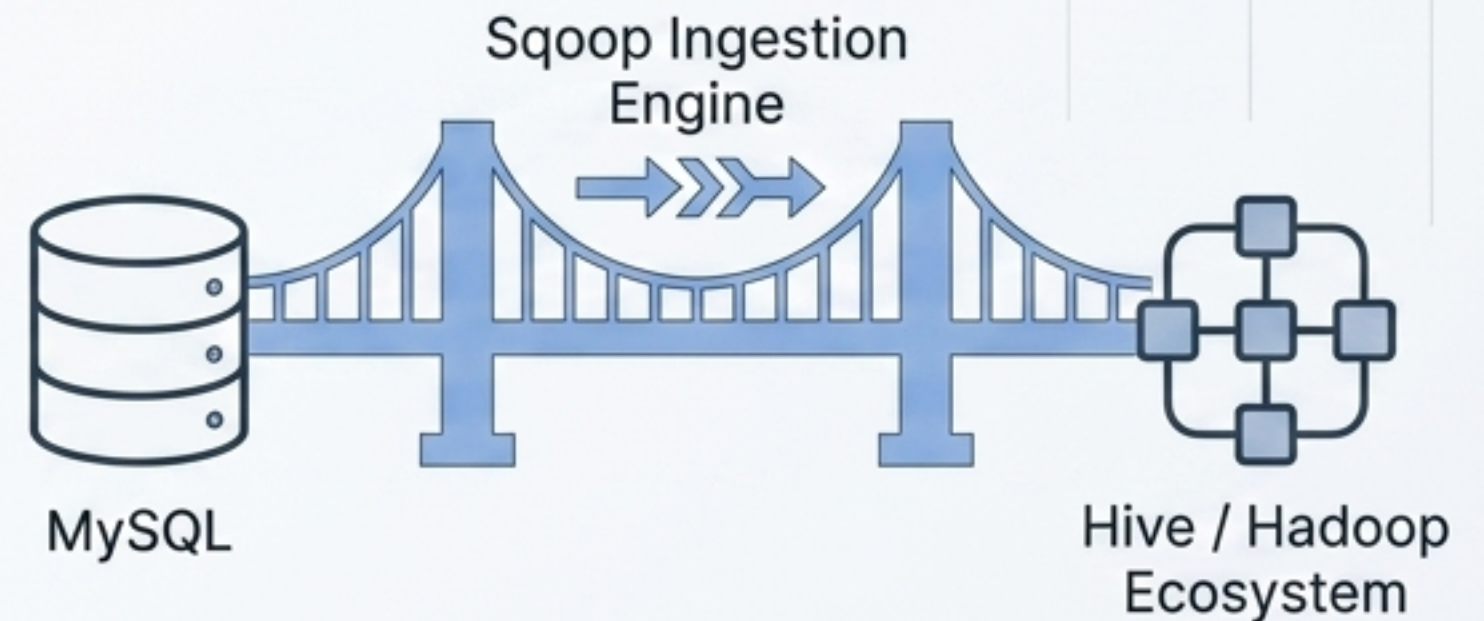
Join Type	Table Sizes	Execution Phase	Key Requirement / Properties
Reduce-Side	Any Size	Reducer	Default behavior; high network & shuffle cost.
Map-Side	1 Small, 1 Large	Mapper	Small table fits in RAM; <code>SET hive.auto.convert.join=true</code>
Sort-Merge	Both Large	Reducer	Both massive tables must be pre-sorted.
SMB Join	Both Large	Mapper / Reducer	Tables must be bucketed & sorted; massively reduces query execution time.

Integration: RDBMS to Hive Pipeline via Sqoop

```
sqoop import \  
  --connect jdbc:mysql://... \  
  --username root --password *** \  
  --table mysql_employee \  
  --hive-import \  
  --create-hive-table \  
  --hive-table employee_migration
```

Crucial flag routing data directly into Hive instead of raw HDFS.

Automatically generates the Hive DDL based entirely on the MySQL schema.



Sqoop abstracts away the manual DDL table creation and MapReduce loading processes, establishing an automated pipeline from RDBMS directly to Hive.